

KAIST CS220

Programming Principles

Instructor: Jeehoon Kang (<https://cp.kaist.ac.kr>)

Foreword

Abstraction, Simple and Relevant

- **Abstraction: meaning given to phenomena**
 - Concept, idea, design, perspective, ... (*Basis of all the studies*)
- **Simplicity: letting the users forget about the details**
 - Map: a simple abstraction of the globe (so useful)
 - 1:1 map: why having such a map after all?
- **Relevance: ... while still working on real-world problems**
 - Head counter: a simple method to ensure everyone is here (so useful)
 - Bean counter: why on earth you want the exact number?
- **Tradeoff: no single abstraction rocks for all the time**
 - Let's measure the height as an abstraction!
 - Ruler: useful for cups, but not for germs
 - Microscope: the other way around

CS: Studying Abstractions of Computational Phenomena

- **Computational phenomena: anything related to computers**
 - Operating systems, networked systems, programming languages, ...
 - Artificial intelligence, machine learning, deep learning, ...
 - Visual computing, social networking, human-computer interaction, ...
 - *“Software is eating the world”: computation is shaking the modern society*
 - *Some are from the nature; the others are **constructed by human***
- **It's abstraction: historical artifacts to understand/exploit them better**
 - System calls, OSI 7 layers, throughput, latency, AST, ...
 - SVM, CNN, Transformer, cross-validation, UI, UX, ...
 - *(Traditions from Mathematics and Electrical Engineering)*

Programming: All of Human-Computer Interaction

- **Very diverse: circuit, programming language, library, user story, ...**
 - Python, R, C, C++, C#, Java, Scala, Clojure, Kotlin, Rust, OCaml, ...
 - LaTeX, Scratch, Blockly, Assembly, Verilog, domain-specific languages, Unity, 아하!, chat bots, calculator, spreadsheet, kiosk, ...
 - *(Difficult to distinguish traditional and modern programming languages)*
- **Simplicity: you don't need to understand implementation details**
 - C: don't even think about the CPU micro-architecture!
- **Relevance: ... while achieving the goal in the real world**
 - C: you can execute any C programs on CPUs!

Two Perspectives on Studying Programming Abstractions

- **Human-centric: programming exists to express human's idea** 
 - Key question: how to express this idea effectively?
 - Idea: constructive Mathematics (value, function, series, ...)
- **Machine-centric: programming exists to run machines**
 - Key question: how to run this machine effectively?
 - Machine: CPU, DRAM, SSD, accelerators, Ethernet, Infiniband, ...
- **Both perspectives are valuable.**
 - CS220: human-to-machine approach
 - CS230: machine-to-human approach
 - Combined together: balanced view on computing

Human-to-machine Approach to Programming

- **Problem definition: formulating real-world problems mathematically**
 - We want a function that determines whether an element is in the array.
- **Solution: designing a mathematical procedure to solve the problem**
 - An element X is in an array L if and only if (exists i , $X = L[i]$).
- **Implementation: expressing the procedure in a programming language**
 - ```
fn find(x: usize, l: Array<usize>) -> bool {
 for (i = 0; i < l.size(); ++i) {
 if (x == l[i]) return true;
 }
 false
}
```
- ... efficiently even for increasingly complex computational phenomena

**This course: learning the expression toolkit**

# Key Contents

- **Part 1: CS101 Review in the Rust programming language**
  - Basic programming skills: using value, function, if-then-else, loop, ...
  - Rust? learning the essence of those skills by comparing Rust and Python
- **Part 2: Type and Correctness**
  - Correctness: in what sense your program works fine?
  - Verification: how to reason about your program's correctness?
  - Type: how to automate such verification?
- **Part 3: Reference and Ownership**
  - Reference: for sharing object
  - Ownership: for reasoning about safe sharing of object
  - Smart pointer: automatic management of pointer ownership
- **Part 4: Function and Iterator**
  - Function as a value: passing a function to another one
  - Iterator: succinctly expressing a program as a series of ideas
- **Part 5: Concurrency and Parallelism**
  - Concurrency: shared resources causing headaches to all programmers (but necessary)
  - Parallelism: distributing tasks to multiple cores *without* explicit consideration of concurrency



# Schedule

- Week 01: introduction
- Week 02: part 1 (CS101 review)
- Week 03: part 1 (CS101 review)
- Week 04: part 1 (CS101 review)
- Week 05: part 2 (type)
- Week 06: part 2 (correctness)
- Week 07: part 2 (type as automatic verification)
- Week 08: mid-term exam
- Week 09: part 3 (reference)
- Week 10: part 3 (ownership)
- Week 11: part 4 (function as a value)
- Week 12: part 4 (combinator)
- Week 13: part 5 (mutability)
- Week 14: part 5 (concurrency)
- Week 15: part 5 (parallelism)
- Week 16: final exam

# Evaluation

- **ABCDF**
- **Programming assignments (40%: 13 times)**
  - Write small Rust programs. We will provide the full test scripts.
  - Submit your solution to <https://gg.kaist.ac.kr> and it will tell you whether your solution is right or wrong. You can submit solutions multiple times until getting right without deduction.
  - If you listen to the lecture carefully, then you can get the full score.
- **Exams (60%: midterm 30%, final 30%)**
  - Many T/F questions, some easy short answer questions, and a few challenging ones.
  - If you listen to the lecture **VERY** carefully, then you can get the full score.
- **Attendance (?%)**
  - Answer attendance quizzes in <https://gg.kaist.ac.kr>.
  - If you're missing 2-3 sessions, you're not going to get any deduction. If you're missing more than 10 sessions, you're likely to get deduction. Between 2-3 and 10, I'll not tell the exact policy.

# Large Language Model Policy

- **Large language models (LLMs): [ChatGPT](#), ...**
  - We assume all of us can use ChatGPT 3.5.
- **You can use LLMs for homework, study, ...**
  - ChatGPT 3.5 helps much for homework and study.
  - Instead, you'll solve more homework problems.
- **You cannot use LLMs for exams.**
- **We'll survey on your experience of LLMs for this course.**

# Using ChatGPT for Homework (1/3)

- ChatGPT can **help** you a lot.
  - It can **generate a code** from the skeleton and description.
  - It can **explain meaning** of a code snippet.
  - It can **suggest fixes** from error messages.

H Please fill in `todo!()`'s of the following Rust program.

```
...

/// Day of week.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum DayOfWeek {
 /// Sunday.
 Sun,
 /// Monday.
 Mon,
 /// Tuesday.
 Tue,
 /// Wednesday.
 Wed,
 /// Thursday.
 Thu,
 /// Friday.
 Fri,
 /// Saturday.
 Sat,
}

/// The next day of week.
///
/// `next_weekday(Thu)` is `Fri`; and `next_weekday(Fri)` is `Mon`.
pub fn next_weekday(day: DayOfWeek) -> DayOfWeek {
 todo!()
}
...
```



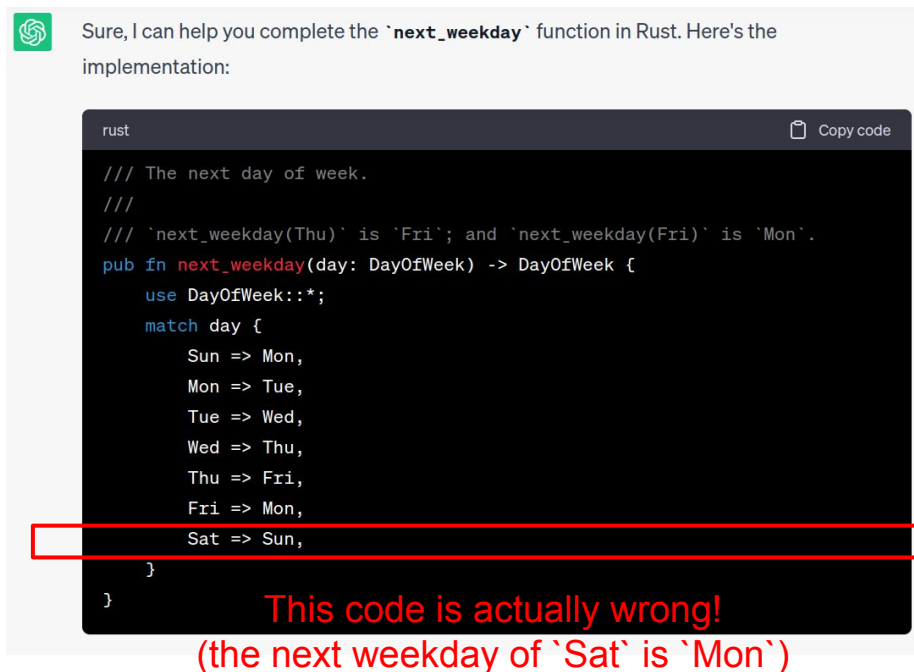
Sure, I can help you complete the `next\_weekday` function in Rust. Here's the implementation:

```
rust Copy code

/// The next day of week.
///
/// `next_weekday(Thu)` is `Fri`; and `next_weekday(Fri)` is `Mon`.
pub fn next_weekday(day: DayOfWeek) -> DayOfWeek {
 use DayOfWeek::*;
 match day {
 Sun => Mon,
 Mon => Tue,
 Tue => Wed,
 Wed => Thu,
 Thu => Fri,
 Fri => Mon,
 Sat => Sun,
 }
}
```

## Using ChatGPT for Homework (2/3)

- ChatGPT **doesn't** solve all your homework.
  - It may **generate wrong code** from skeleton and description.
  - It may **wrongly explain meaning** of a code snippet.
  - It may **suggest wrong fixes** from error messages.



Sure, I can help you complete the `next\_weekday` function in Rust. Here's the implementation:

```
rust Copy code
/// The next day of week.
///
/// `next_weekday(Thu)` is `Fri`; and `next_weekday(Fri)` is `Mon`.
pub fn next_weekday(day: DayOfWeek) -> DayOfWeek {
 use DayOfWeek::*;
 match day {
 Sun => Mon,
 Mon => Tue,
 Tue => Wed,
 Wed => Thu,
 Thu => Fri,
 Fri => Mon,
 Sat => Sun,
 }
}
```

**This code is actually wrong!**  
(the next weekday of `Sat` is `Mon`)

# Using ChatGPT for Homework (3/3)

- You should **refine** its results.
  - Provide failed test case to ChatGPT.
  - Fix on your own.

```
/// The next day of week.
///
/// `next_weekday(Thu)` is `Fri`; and `next_weekday(Fri)` is `Mon`.
pub fn next_weekday(day: DayOfWeek) -> DayOfWeek {
 use DayOfWeek::*;
 match day {
 Sun => Mon,
 Mon => Tue,
 Tue => Wed,
 Wed => Thu,
 Thu => Fri,
 Fri => Mon,
 Sat => Mon, // Saturday's next weekday is Monday.
 }
}
```

# Logistics

- **Homepage:** <https://github.com/kaist-cp/cs220>
  - Read carefully!
  - Announcement in the issue tracker (please watch the repo)
- **Questions:** <https://github.com/kaist-cp/cs220/issues>
  - TA's office hour: 09:15-10:15 am, Friday
- **Attendance check, homework submission:** <https://gg.kaist.ac.kr>
  - I'll create your accounts soon.
- **Honor code:** I assume you signed one for KAIST SoC.

# Part 1:

# CS101 Review in Rust



# KAIST CS101: Introduction to Programming

- <https://cs101.kaist.ac.kr/>, <https://kooc.kaist.ac.kr/cs101>
  - *Basic programming skills in Python*
  - CHAPTER 1. CS101 강의 소개
  - CHAPTER 2. if 조건문과 while 반복문
  - CHAPTER 3. 변수와 기초 자료형
  - CHAPTER 4. 매개 변수와 반환값을 가진 함수
  - CHAPTER 5. 지역/전역 변수와 그래픽 객체들
  - CHAPTER 6. 시퀀스: 리스트, 문자열, 튜플
  - CHAPTER 7. 다양한 자료구조 ( 문자열, 집합, 사전)
  - CHAPTER 8. 자료구조를 활용한 이미지 및 텍스트 프로세싱
  - CHAPTER 9. 객체: 생성과 속성
  - CHAPTER 10. 프로그램 작성 고급 기술
- **CS220 assumes basic programming skills (not necessarily in Python).**

# What's Rust? Why Rust?

- **What: a modern programming language**
  - Supporting various domains (from systems to applications)
  - Decent tools for build (cargo), lint (rustfmt, clippy), IDE (VSCode)
- **Why 1: learning multiple programming languages**
  - Learning the similarities and differences of programming languages
  - Learning what's essential and what's not about programming
- **Why 2: enjoying the modern language's benefits**
  - Productive
  - Type-safe (Part 2), functional (Part 4), parallelism-friendly (Part 5)
  - Safe pointer programming (Part 3)

# Agenda

- **Reading the Rust Book:** <https://doc.rust-lang.org/book/>
- **Extremely well-written; meant to be self-contained**
  - You can also read the book from the beginning to the end on your own.
- **You'll read the book for CS220 Part 1: CS101 Review.**
  - I'll walk through the book and discuss a few interesting points.
  - Chapters 1-3, 5-9, 11-12
  - (the other chapters for the later parts)
- **If it's too easy for you, please wait for the Parts 2-5...**
- **Assignment-first approach**
  - Before learning materials, I'll give you relevant assignment first.

# Assignment 01 (for Chapter 1)

- **Description:** [here](#)
  - Purpose: bringing up Rust development environment
- **Deadline: 10/04 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lives,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/131/>
  - Submit your assignment01.zip.
  - You can submit your solution as many time as you wish.
  - Only those submissions with “Valid?” being “Yes” are considered.
  - Only the last valid submission is considered.
  - A valid submission may not get the full score; see the end of the log.
  - No “hidden test cases.”

# Preparing Rust Development Environment

- **Option 1: Installing Rust in the local machine:** <https://rustup.rs/>
  - Choose this option if you know what you're doing
- **Option 2: Using the provided development server**
  - <https://cloud.fearless.systems>
  - <https://github.com/kaist-cp/cs220#tools>
- **Forking the CS220 repository:** <https://github.com/kaist-cp/cs220>
  - CRUCIAL: don't fork it as a public repository! I'll remove public forks.
  - [How to fork a public repository as private?](#)

# Chapter 1: Getting Started

- Reading assignment: [Chapter 1](#) (by 9/8 Fri)
  - (more in later)
- Notes
  - Chapter 1.1: we recommend you to use the provided dev server.
  - Chapter 1.3: repository structure is better to be fixed as a convention.
    - Don't try to be too creative when it's really not worth it, e.g., indent size.
    - Focus on your problem domain, e.g., printing "hello world."
  - Chapter 1.3: development tools are extremely important for *code reuse*.
    - Rust: cargo for build/doc, rustfmt/clippy for lint, rust-analyzer for IDE interop, <https://crates.io> for library/binary repository, ...
    - C/C++: lacking standard dev tools
    - *"Don't reinvent the wheel:"* the software engineering practice 101

# Chapter 2: Guessing Game

- Reading assignment: [Chapter 2](#) (by 9/8 Fri)
- Notes
  - Implement the guessing game by copying-and-pasting the code.
    - Copying is an extremely efficient way to learn programming.
  - Compiler is the best tutor: read error messages carefully!
  - Immutable/mutable references (&, &mut)
    - The “permission” to access the referent, read-only/read-write
    - (more in later)
  - `cargo add rand`
    - The easiest way to add `rand` as a dependency

# Assignment 02 (for Chapters 3, 5)

- **Description:** [here](#)
  - Purpose: mastering common programming concepts (1/2)
- **Deadline: 10/04 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lives,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/106/>
  - Submit your assignment02.zip.



# Chapter 3: Common Programming Concepts

- Reading assignment: [Chapter 3](#) (by 9/15 Fri)
- Notes
  - 3.1: Immutability by default to reduce bugs.
    - When variable shadowing suffices, use it instead of mutable variables.
  - 3.2: Type to reduce bugs.
    - `let x: u64 = 42; x` will never contain a string, for example.
  - 3.2: Integer types have minimum/maximum bounds for efficiency.
    - `wrapping_add`, `checked_add`, `overflowing_add`, `saturating_add`, ...
    - [https://docs.rs/num-bigint/latest/num\\_bigint/](https://docs.rs/num-bigint/latest/num_bigint/) for Python-style “big ints.”
  - 3.2: Prefer structs w/ names to tuples w/o names.
    - Give semantic meaning to fields.
  - 3.2: Buffer overrun *always* incurs runtime error by default.
    - In C, it is *sometimes* silently ignored (leading to a super obscure bugs).
  - 3.3: Parameter types are not inferred for documentation purposes.
  - 3.4: Comments should be about policy (not mechanism).
  - 3.5: Prefer container iteration (w/ more semantics) to plain loops.

# Chapter 5: Using Structs to Structure Related Data

- Reading assignment: [Chapter 5](#) (by 9/15 Fri)
- Notes
  - Structs are encapsulation: hiding implementation, exposing API.
    - We prefer narrow API w/ deep functionality, e.g., simple map API w/ extremely complicated concurrent hash table impl.
  - Don't use tuple structs (unless you have a very good reason).
    - Names serve as documentation.
  - Methods are just a syntactic sugar of functions.
    - Following the OOP tradition of discerning a single “receiver” of a function...
    - But it doesn't need to be that way (cf. Common Lisp Object System)
  - `&self`, `&rect` for “referencing” the `self`, `rect` objects (to avoid copying)
  - `impl Rectangle` can even be in a different file (in the same crate)!

# Assignment 03 (for Chapters 6-9)

- **Description:** [here](#)
  - Purpose: mastering common programming concepts (2/2)
- **Deadline: 10/04 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lives,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/132/>
  - Submit your assignment03.zip.

# Chapter 7: Managing Growing Projects with Packages, Crates, and Modules

- Reading assignment: [Chapter 7](#) (by 9/23 Fri)
- Notes
  - Policy: how to organize and refer entities in a large codebase?
    - Organization: “Grouping Related Code in Modules”
    - Reference: relative vs. absolute
    - Visibility: hide things as much as possible
    - Example: the `cs220` repository
  - Mechanism: let `rustfmt` and `rust-analyzer` do their work.
    - Don’t think too much about code formatting (e.g., path resolution). Follow the widely-used conventions.
    - Don’t try to hand-optimize formatting, e.g., the use lists.
  - Tool: let IDEs do their work.
    - VScode > “Preferences: Open User Settings” > check “Format on Save”

# Chapter 6: Enums and Pattern Matching

- Reading assignment: [Chapter 6](#) (by 9/23 Fri)
  - Optional: [Chapter 18](#) (strongly recommended: it'll make your life easier!)
- Notes
  - struct : enum = and : or
    - A struct value has *all* fields; an enum value is *some* variant.
    - An enum value is always of a single variant (≠ union in C/C++)
  - Enum makes a *closed-world* assumption.
    - E.g., an IP protocol is either V4 or V6 (**but nothing else**).
    - Benefit: pattern match mandates you to handle all possibilities.
      - For `Option<T>`, an explicit consideration for `Some(t)` and `None` is necessary.
  - Enums are heavily optimized by the compiler!
    - `Option<u64>` has the same size as `u64`.
  - Don't think too much on `match`, if `let`, `{}` or `not`, ... Just run linters.
    - `Rustfmt` and `clippy` will automatically fix your code style.
  - Prefer early return!
    - It greatly simplifies the organization of code w/ complex control flow.
    - Have a look at <https://docs.rs/etrace/latest/etrace/> for advanced early return patterns.

# Chapter 8: Common Collections

- Reading assignment: [Chapter 8](#) (by 9/23 Fri)
- Notes
  - Rust mandates you to think of data structure implementation details.
    - ??? “adding a new element onto the end of the vector might require allocating new memory and copying the old elements to the new space”
    - ??? “UTF-8 encoded”
    - ??? “SipHash that can provide resistance to Denial of Service (DoS) attacks involving hash tables”
    - ... but as little as possible.
  - Rust Strings are not just byte arrays: they are UTF-8 encoded chars.
  - To access Rust maps, try the entry API before the others.

# Chapter 9: Error Handling

- Reading assignment: [Chapter 9](#) (by 9/23 Fri)
- Notes
  - Unrecoverable error: `panic!`
    - When you don't know what to do.
  - Recoverable error: `Result<T, E>`
    - When you know what to do.
    - Get used to error handling/propagation methods (e.g., `unwrap_or_else`).
    - Get used to the `?` operator.
  - Address and forget: don't leave error handling as future work.
    - Unhandled errors will bite you later.

# Assignment 04 (for Chapters 11-12)

- **Description:** [here](#)
  - Purpose: applying what we've learned to a realistic project
- **Deadline: 10/11 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, "lives," ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/134/>
  - Submit your assignment04.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says "Yes" for the "Success?" column, you're good.
    - No "hidden test cases."



# Chapter 11: Writing Automated Tests

- Reading assignment: [Chapter 11](#) (by 9/30 Fri)
- Notes
  - The entirety of this chapter is useful for debugging. Read carefully.
  - Test function example: the `cs220` repository
  - Q: when to write a unit/integration tests?
    - Before writing the code?
    - After writing the code?
    - After detecting some anomalies?
    - Never?
  - A: when it's beneficial: guiding the implementation, fault localization, ...
  - Main cannot be tested; place almost everything in lib.

# Chapter 12: An I/O Project: Building a Command Line Program

- Reading assignment: [Chapter 12](#) (by 9/30 Fri)
- Notes
  - We won't walk through the chapter in class thoroughly.
    - Still, it's strongly recommended for you to copy-and-paste the textbook for a hands-on experience.
  - Use [the clap crate](#) for argument parsing.
    - It already solved the problem for us once and for all...
    - Let's avoid environment variables (at least at the beginning...)
  - Let's ignore "lifetimes" for now.
    - Just type as the book says...

# Part 2:

# Type and Correctness

# Assignment 05 (for program correctness and logic)

- **Description:** [here](#)
  - Purpose: What's program correctness? How to verify it?
- **Deadline: 10/11 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, "lives," ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/135/>
  - Submit your assignment05.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says "Yes" for the "Success?" column, you're good.
    - No "hidden test cases."

# Program Logic: Proof Technique for Correctness

- **Proving a program's correctness w/ deductive reasoning**
  - Understanding program correctness as mathematical formula
  - Example: proving the correctness of binary search!
- **Tool:** <https://www.why3.org/try/>
  - Choose the “whyml” dialect.
- **Examples**
  - <https://github.com/kaist-cp/cs220/tree/main/assets/why3>
  - <https://gitlab.inria.fr/why3/why3/-/tree/master/examples>
- **Invariant: statement that holds at that line**
  - Pre/post-condition: a function's specification
  - Loop invariant: the “inductive hypothesis” for the loop

# Program Correctness: a Multi-faceted Concept

- **Functionality**

- Definition: suppose a function's pre-condition holds. After executing the function, its post-condition should hold.
- Reason: each line “preserves” the invariant.

- **Termination**

- Definition: a function's execution should terminate.
- Reason: for each loop iteration, “something” should be decreased.  
`// variant { something }`

- **Safety**

- Definition: a function's execution does not invoke “undefined behavior.”  
(more on side-channel attacks at IS522: Introduction to System Security)
- Reason: each line is well-behaved assuming the invariant.

- **(More on program logics at CS520: Theory of Programming Languages)**

## Type: Invariant Representation supporting Automated Verification

- **Representation:** “At line 10, variable  $x$  should be 64-bit unsigned int.”
  - $\{ \text{has\_type}(x, \text{u64}) \wedge \dots \}$   
L10: ...  
 $\{ \dots \}$
  - Implicit: you don't actually write down such invariants.
- **Efficiency:** we can generate/check such invariants automatically.
  - “Type checking algorithm”: usually  $\text{sub-}O(n^2)$
  - More in CS320: Programming Languages
- **Soundness:** if a program is type-checked, the type invariants hold.
  - More complex types  $\Rightarrow$  more expressive invariants w/ difficult proofs
  - Dream: automatic verification of *all* programs, even w/ loops?

# Assignment 06 (for generic types and traits)

- **Description:** [here](#)
  - Purpose: How far we can automate program verification w/ types?
- **Deadline: 10/18 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lives,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/136/>
  - Submit your assignment06.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says “Yes” for the “Success?” column, you’re good.
    - No “hidden test cases.”



# Chapter 10: Generic Types, Traits, ~~and Lifetimes~~

- Reading assignment: [Chapter 10](#) (except 10.3, by 10/14 Fri)
- Notes
  - The motivation of generic type and trait is generalization (deduplication).
    - “Polymorphism” in other words
  - When to perform generalization?
    - When necessary: to avoid waste of time
    - From the beginning: abstract thinking for general things helps!
    - *It really depends.*
  - Impl Trait in the return position is “existential” type.
    - The return type is hidden to the function’s client (for encapsulation purposes).
    - From the client’s point of view, the return type is existential.

# **MIDTERM EXAM**

**Everything you've learned so far**

# Part 3:

## Reference and Ownership

# Assignment 07 (for reference and ownership)

- **Description:** [here](#)
  - Purpose: How far we can automate program verification w/ types?
- **Deadline: 11/08 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lives,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/137/>
  - Submit your assignment07.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says “Yes” for the “Success?” column, you’re good.
    - No “hidden test cases.”

# Chapter 4: Understanding Ownership (1/2)

- Reading assignment: [Chapter 4](#) (by 10/28 Fri)
- Notes
  - Memory-safe languages should reason about each reference's safety.
    - Reason: if the referenced object is gone, the reference is dangling! unsafe!
    - Python: it guarantees referenced objects are never gone. Simple but costly.
    - C: it guarantees nothing. Complex and dangerous.
  - Rust uses ownership types to reason about each reference's safety.
    - Only those references with (borrowed) ownerships are safe to deref.
    - E.g., "this reference is safe to deref from here to here (in this lifetime)."
    - E.g., "since ref1 is no longer valid, ref2 can modify it."
    - Type's benefit: safety & low-cost

# Chapter 4: Understanding Ownership (2/2)

- Reading assignment: [Chapter 4](#) (by 10/28 Fri)
- Notes
  - Aliasing  $\oplus$  Mutation: the first rule of ownership
    - Aliasing: referencing at the same time
    - Mutation: at least one reference is used to modify the object
    - Example: single mutable reference; multiple immutable references
  - Aliasing AND Mutation: prevented by the type checker
    - Reason: ref. A may dealloc the object so that ref. B may become dangling.
    - Compile-time reasoning of concurrency w/ lifetime annotations
      - 'lft: an over-approximation of a reference's actual lifetime
    - Compile-time reasoning of mutability w/ explicit &mut annotations
      - &: read-only; &mut: read-write
- Two more things
  - [Chapter 5.1 "Ownership of Struct Data"](#)
  - [Chapter 10.3: Validating References with Lifetimes](#)

# Assignment 11 (for smart pointers)

- **Description:** [here](#)
  - Purpose: familiarizing with smart pointers
- **Deadline: 12/05 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lifes,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/141/>
  - Submit your assignment11.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says “Yes” for the “Success?” column, you’re good.
    - No “hidden test cases.”

# Chapter 15: Smart Pointers

- Reading assignment: [Chapter 15](#) (by 11/25 Fri)
- Notes
  - Deref/Drop for ergonomic use of APIs (they are language features!)
  - Box for (1) large data and (2) recursive types
  - Rc for sharing data (but not mutation)
  - RefCell for enforcing the A $\oplus$ M principle *at runtime*
    - Interior mutability: enveloping potentially unsafe code within safe API
  - Even, a garbage collector: <https://github.com/Manishearth/rust-gc>

Here is a recap of the reasons to choose `Box<T>`, `Rc<T>`, or `RefCell<T>`:

- `Rc<T>` enables multiple owners of the same data; `Box<T>` and `RefCell<T>` have single owners.
- `Box<T>` allows immutable or mutable borrows checked at compile time; `Rc<T>` allows only immutable borrows checked at compile time; `RefCell<T>` allows immutable or mutable borrows checked at runtime.
- Because `RefCell<T>` allows mutable borrows checked at runtime, you can mutate the value inside the `RefCell<T>` even when the `RefCell<T>` is immutable.

Mutating the value inside an immutable value is the *interior mutability* pattern. Let's look at a situation in which interior mutability is useful and examine how it's possible.



# Part 4:

# Function and Iterator

# Assignment 08 (for first-class functions)

- **Description:** [here](#)
  - Purpose: Understanding that functions are also values
- **Deadline: 11/15 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lives,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/138/>
  - Submit your assignment08.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says “Yes” for the “Success?” column, you’re good.
    - No “hidden test cases.”

# Chapter 13.1: Closures

- Reading assignment: [Chapter 13.1](#) (by 11/11 Fri)
- Notes
  - Closure = anonymous function = lambda function = first-class function
  - Regarding reuse
    - “Higher-order function”: function that takes function parameter
      - [unwrap\\_or\\_else](#)
      - [sort\\_by\\_key](#)
    - Read Rust standard library documentation for [Option](#) and [Result](#).
  - Regarding ownership
    - “Unlike [ordinary] functions, closures can capture values from the scope in which they’re defined.”
    - Fn vs. FnOnce vs. FnMut
    - Fn for maximal reuse; FnMut for maximal capability

# Assignment 09 (for iterators, 1/2)

- **Description:** [here](#)
  - Purpose: Succinctly expressing programs as a series of ideas
- **Deadline: 11/29 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lifes,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/139/>
  - Submit your assignment09.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says “Yes” for the “Success?” column, you’re good.
    - No “hidden test cases.”

# Assignment 10 (for iterators, 2/2)

- **Description:** [here](#)
  - Purpose: Succinctly expressing programs as a series of ideas
- **Deadline: 11/29 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lifes,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/140/>
  - Submit your assignment10.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says “Yes” for the “Success?” column, you’re good.
    - No “hidden test cases.”

# Chapters 13.2-13.4: Iterators

- Reading assignment: [Chapter 13.2](#), [Chapter 13.3](#), [Chapter 13.4](#) (by 11/25 Fri)
- Notes
  - Many programs are transformer from request stream to response stream
    - Web server, GPU kernel, ...
  - Iterator provides vocabulary for such programs in higher-order functions
    - map, filter, filter\_map, ...
  - A primary application of first-class functions
    - Maximal reusability w/ minimal user-custom code
  - Iterator almost always simplifies the structure of programs
    - Even for hardware designs! [ShakeFlow](#) (ASPLOS 2023), [HazardFlow](#) (PLDI 2024)

# Documentation for Iterators

- **Reading assignment (by 11/25 Fri)**
  - <https://doc.rust-lang.org/rust-by-example/trait/iter.html>
  - <https://doc.rust-lang.org/stable/std/iter/>
  - <https://docs.rs/itertools/latest/itertools/>
- **Notes**
  - Think streams before elements.
  - Think idea first and search for iterators later.
  - Don't need to memorize all iterators; search them with function types.

# Part 5:

# Concurrency and Parallelism



# Assignment 12 (for concurrency)

- **Description:** [here](#)
  - Purpose: What's involved with multi-threaded programs?
- **Deadline: 12/20 (Fri) 23:59:58 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, "lives," ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/142/>
  - Submit your assignment12.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says "Yes" for the "Success?" column, you're good.
    - No "hidden test cases."

# Chapters 16: Fearless Concurrency

- **Reading assignment (by 12/09 Fri)**
  - [Chapter 16](#)
  - <https://rust-lang-nursery.github.io/rust-cookbook/concurrency.html>
- **Notes**
  - By default, Rust enforces the the  $A \oplus M$  principle.
    - The same for concurrency!
  - Shared concurrent state (e.g., channel) is fundamentally A&M.
    - Aliased: multiple threads are accessing the same channel.
    - Mutated: multiple threads are sending/receiving messages via the channel.
  - Resolving the conflict with interior mutability
    - Mutex
    - Channel
    - Concurrent data structures
  - Extensible support for shared concurrent state with Send/Sync traits

# Assignment 13 (for data parallelism)

- **Description:** [here](#)
  - Purpose: Streams are easier to parallelize!
- **Deadline: 12/20 (Fri) 23:59:59 (KST)**
  - CRUCIAL: there will be no extensions, late submissions, “lives,” ...
- **Submission:** <https://gg.kaist.ac.kr/assignment/143/>
  - Submit your assignment13.zip.
  - You can submit your solution as many time as you wish.
  - If any of the rows says “Yes” for the “Success?” column, you’re good.
    - No “hidden test cases.”

# Rayon: Data Parallelism Library

- Reading assignment (by 12/09 Fri)
  - <https://docs.rs/rayon/latest/rayon/>, especially:
    - <https://docs.rs/rayon/latest/rayon/iter/trait.ParallelIterator.html>
    - <https://docs.rs/rayon/latest/rayon/iter/trait.IndexedParallelIterator.html>
    - [https://docs.rs/rayon/latest/rayon/slice/trait.ParallelSliceMut.html#method.par\\_sort](https://docs.rs/rayon/latest/rayon/slice/trait.ParallelSliceMut.html#method.par_sort)
- Notes
  - Your program is *seamlessly* parallelized: adding a few words is enough!
    - The library does all the hard work.
  - ... as far as your program satisfies Send/Sync requirements.
  - (How to actually parallelize the program? Go to CS431...)

# Afterword

## Recall:

# Two Perspectives on Studying Programming Abstractions

- **Human-centric: programming exists to express human's idea** 
  - Key question: how to express this idea effectively?
  - Idea: constructive Mathematics (value, function, series, ...)
- **Machine-centric: programming exists to run machines**
  - Key question: how to run this machine effectively?
  - Machine: CPU, DRAM, SSD, accelerators, Ethernet, Infiniband, ...
- **Both perspectives are valuable.**
  - CS220: human-to-machine approach
  - CS230: machine-to-human approach
  - Combined together: balanced view on computing

# Recall:

## Key Contents

- **Part 1: CS101 Review in the Rust programming language**
  - Basic programming skills: using value, function, if-then-else, loop, ...
  - Rust? learning the essence of those skills by comparing Rust and Python
- **Part 2: Type and Correctness**
  - Correctness: in what sense your program works fine?
  - Verification: how to reason about your program's correctness?
  - Type: how to automate such verification?
- **Part 3: Reference and Ownership**
  - Reference: for sharing object
  - Ownership: for reasoning about safe sharing of object
  - Smart pointer: automatic management of pointer ownership
- **Part 4: Function and Iterator**
  - Function as a value: passing a function to another one
  - Iterator: succinctly expressing a program as a series of ideas
- **Part 5: Concurrency and Parallelism**
  - Concurrency: shared resources causing headaches to all programmers (but necessary)
  - Parallelism: distributing tasks to multiple cores *without* explicit consideration of concurrency

# Instructor's Hope

- **You've found the programming idioms interesting.**
  - ... and useful.
- **You're applying these idioms.**
  - ... to courses, to open source projects, and in your future career.
- **You're going to take further programming courses.**
  - [CS320: Programming Languages](#)
  - ... and maybe [CS420: Compiler Design](#)?
  - ... and maybe [CS431: Concurrent Programming](#)?
  - ... and maybe CS492: Program Reasoning?
  - ... and maybe CS524: Program Analysis?



## If you're interested in programming research...

- Consider an internship at [Concurrency and Parallelism Laboratory](#)!
  - [Contact me](#) and [join the laboratory](#) any time!
- We're designing and verifying concurrent/parallel systems.
  - *Hardware circuit designs*: circuit is just a massively parallel program.
  - *Concurrent data structures*: we're verifying their correctness.
  - *Emerging hardware models*: we're understanding NPUs, persistent memory, SSD controllers, ...
  - *Rust OS*: we're verifying OS with low cost by leveraging Rust types.

Thank you for  
taking this course!

# **FINAL EXAM**

**Everything you've learned after midterm**

# Garbage Slides

# Schedule

- Week 01: foreword (introduction and tool setup)
- Week 02: part 1 (the Rust book)
- Week 03: part 1 (the Rust book)
- Week 04: part 1 (the essence of programming concepts)
- Week 05: part 2 (type)
- Week 06: part 2 (correctness and verification)
- Week 07: part 2 (type as an automated verification), exam study guide
- Week 08: mid-term exam
- Week 09: part 3 (reference and ownership)
- Week 10: part 3 (type-checking ownership)
- Week 11: part 4 (function as a value)
- Week 12: part 4 (combinator and dataflow applications)
- Week 13: part 5 (mutability and interior mutability)
- Week 14: part 5 (concurrency)
- Week 15: part 5 (data parallelism), exam study guide
- Week 16: final exam